

ЛАБОРАТОРНА РОБОТА № 3

Хід роботи:

Завдання 1: Підготувати датасет з двома класами зображень (зображення котів та собак). Зображення завантажені з мережі та збережені у відповідних папках.

Завдання 2: Завантажити зображення двох класів на Google Drive: папка Cats (519 зображень) та папка Dogs (530 зображень).

Завдання 3: Підключити Google Drive до Google Colab Notebook за допомогою `drive.mount('/content/drive')`.

Завдання 4: Зчитати дані картинки cats та відобразити зразки зображень обох класів у Google Colab.

Завдання 5: Перевірити розширення картинок, видалити усі окрім 'jpeg', 'jpg', 'bmp', 'png'. Залишено 1049 зображень з розширенням jpeg.

Завдання 6: Створити датасет з картинок за допомогою `tf.keras.utils.image_dataset_from_directory` з розміром зображень 128x128 та `batch_size=8`.

Завдання 7: Створити `numpy_iterator` з датасету за допомогою методу `as_numpy_iterator` та відобразити перший батч.

Завдання 8: Нормалізувати дані за допомогою `layers.Rescaling(1.0/255)`: значення пікселів переведено з діапазону [0, 255] у [0.0, 1.0]. Також додано аугментацію: `RandomFlip`, `RandomRotation`, `RandomZoom`, `RandomContrast`.

Завдання 9: Розділити дані на тренувальні (70% - 729 зразків), валідаційні (15% - 152 зразки) та тестувальні (15% - 168 зразків).

Завдання 10: Реалізувати згорткову нейронну мережу (CNN) з шарами `Conv2D` (32, 64, 128, 256 фільтрів), `BatchNormalization`, `MaxPooling2D`, `Flatten`, `Dense` (256, 64, 1). Оптимізатор `Adam`, `loss binary_crossentropy`.

Завдання 11: Натренувати нейронну мережу (15 епох з `EarlyStopping` та `ReduceLROnPlateau`) та протестувати: точність на тестовій вибірці - 100.00%.

					ДУ «Житомирська політехніка». 26.121.20.000 – Лр3			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Рижченко Я.В.			Звіт з лабораторної роботи		Літ.	Арк.
Перевір.		Годлевський Ю.О.						1
Керівник								9
Н. контр.							ФІКТ Гр. ІПЗ-23-1	
Зав. каф.								

Лістинг програми:

```
import os
import zipfile
import shutil
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential
from pathlib import Path

print(f'TensorFlow: {tf.__version__}')
print(f'GPU: {tf.config.list_physical_devices("GPU")}')

# Step 1. Dataset preparation
from google.colab import drive
drive.mount('/content/drive')

DRIVE_BASE = Path('/content/drive/MyDrive/Neural Networks')
DATASET_DIR = Path('/content/dataset')

if DATASET_DIR.exists():
    shutil.rmtree(DATASET_DIR)

for cls in ['cats', 'dogs']:
    dest = DATASET_DIR / cls
    dest.mkdir(parents=True, exist_ok=True)
    zips = list((DRIVE_BASE / cls.capitalize()).glob('*.*.zip'))
    if not zips:
        print(f'no zip found for {cls}')
        continue
    with zipfile.ZipFile(zips[0], 'r') as z:
        z.extractall(dest)
    for sub in [p for p in dest.iterdir() if p.is_dir()]:
        for img in sub.iterdir():
            img.rename(dest / img.name)
        sub.rmdir()
    print(f'{cls}: {len(list(dest.glob("*.*")))} images')

# Step 3-4. Reading and displaying images
VALID_EXTENSIONS = ('.jpg', '.jpeg', '.png', '.bmp', '.JPG', '.JPEG')
sample_cats = [p for p in (DATASET_DIR / 'cats').iterdir() if p.suffix in VALID_EXTENSIONS]
sample_path = sample_cats[0]
sample_img = mpimg.imread(str(sample_path))
print(f'file: {sample_path.name}')
print(f'shape: {sample_img.shape}')
print(f'dtype: {sample_img.dtype}')
print(f'range: {sample_img.min()} - {sample_img.max()}')

fig, axes = plt.subplots(2, 4, figsize=(14, 7))
fig.suptitle('Dataset samples', fontsize=16)
for row, cls in enumerate(['cats', 'dogs']):
    images = [p for p in (DATASET_DIR / cls).iterdir() if p.suffix in VALID_EXTENSIONS][:4]
```

		Риженко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Лр3	Арк.
		Годлевський Ю.О				
Змн.	Арк.	№ докум.	Підпис	Дата		2

```

for col in range(4):
    ax = axes[row, col]
    if col < len(images):
        img = mpimg.imread(str(images[col]))
        ax.imshow(img)
        ax.set_title(f'{cls.upper()}\n{img.shape}', fontsize=10)
    ax.axis('off')
plt.tight_layout()
plt.show()

# Step 5. File extension filtering
ALLOWED_EXTENSIONS = {'jpeg', 'jpg', 'bmp', 'png'}
removed = 0
kept = 0
ext_counter = {}
for cls in ['cats', 'dogs']:
    for f in list((DATASET_DIR / cls).glob('*')):
        if not f.is_file():
            continue
        ext = f.suffix.lower().lstrip('.')
        ext_counter[ext] = ext_counter.get(ext, 0) + 1
        if ext not in ALLOWED_EXTENSIONS:
            f.unlink()
            removed += 1
        else:
            kept += 1
print(f'extensions: {ext_counter}')
print(f'kept: {kept}, removed: {removed}')

# Step 6. Dataset via image_dataset_from_directory
IMG_SIZE = (128, 128)
BATCH_SIZE = 8
SEED = 42

full_dataset = tf.keras.utils.image_dataset_from_directory(
    str(DATASET_DIR),
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=True,
    seed=SEED,
    label_mode='binary'
)
class_names = full_dataset.class_names
print(f'classes: {class_names}')
print(f'batches: {len(full_dataset)}')

# Step 7. Numpy iterator
numpy_iter = full_dataset.as_numpy_iterator()
first_batch = next(numpy_iter)
images_np, labels_np = first_batch
print(f'shape: {images_np.shape}')
print(f'dtype: {images_np.dtype}')
print(f'range: [{images_np.min():.1f}, {images_np.max():.1f}]')
print(f'first 8 labels: {labels_np[:8].flatten().astype(int)}')

# Step 8. Normalisation

```

		Рижченко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Лр3	Арк.
		Годлевський Ю.О				
Змн.	Арк.	№ докум.	Підпис	Дата		3

```

normalization_layer = layers.Rescaling(1.0 / 255)
normalized_dataset = full_dataset.map(
    lambda x, y: (normalization_layer(x), y)
)
augmentation_layer = tf.keras.Sequential([
    layers.RandomFlip('horizontal'),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
    layers.RandomContrast(0.1),
], name='augmentation')

# Step 9. Train / val / test split (70% / 15% / 15%)
total_batches = len(normalized_dataset)
train_size = int(total_batches * 0.70)
val_size = max(1, int(total_batches * 0.15))
test_size = max(1, total_batches - train_size - val_size)
train_size = total_batches - val_size - test_size
print(f'total: {total_batches} batches -> train: {train_size}, val: {val_size}, test: {test_size}')

AUTOTUNE = tf.data.AUTOTUNE
normalized_dataset = normalized_dataset.cache().shuffle(
    buffer_size=total_batches, seed=SEED, reshuffle_each_iteration=False
)
train_ds = normalized_dataset.take(train_size).shuffle(1000, reshuffle_each_iteration=True).map(
    lambda x, y: (augmentation_layer(x, training=True), y), num_parallel_calls=AUTOTUNE
).prefetch(AUTOTUNE)
remaining = normalized_dataset.skip(train_size)
val_ds = remaining.take(val_size).prefetch(AUTOTUNE)
test_ds = remaining.skip(val_size).prefetch(AUTOTUNE)

# Step 10. CNN architecture
model = Sequential([
    layers.Input(shape=(128, 128, 3)),
    layers.Conv2D(32, (3, 3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Conv2D(64, (3, 3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Conv2D(128, (3, 3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Conv2D(256, (3, 3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Flatten(),
    layers.Dense(256),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dropout(0.5),
    layers.Dense(64),
    layers.BatchNormalization(),
])

```

		Рижченко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – ЛрЗ	Арк.
		Годлевський Ю.О				
Змн.	Арк.	№ докум.	Підпис	Дата		4

```

layers.Activation('relu'),
layers.Dense(1, activation='sigmoid'),
], name='CNN_Cats_vs_Dogs')

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
model.summary()

# Step 11. Training
EPOCHS = 15
callbacks = [
    keras.callbacks.ModelCheckpoint(
        'best_model.keras',
        monitor='val_accuracy',
        save_best_only=True,
        verbose=1
    ),
    keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=4,
        restore_best_weights=True,
        verbose=1
    ),
    keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=2,
        verbose=1
    ),
]
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=callbacks,
    verbose=1
)

# Evaluation
test_loss, test_accuracy = model.evaluate(test_ds, verbose=1)
print(f'test accuracy: {test_accuracy * 100:.2f}% loss: {test_loss:.4f}')

# Classification report + confusion matrix
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

y_true_list = []
y_pred_list = []
for images_batch, labels_batch in test_ds:
    preds = model.predict(images_batch, verbose=0)
    y_pred_list.extend((preds > 0.5).astype(int).flatten())
    y_true_list.extend(labels_batch.numpy().astype(int).flatten())

```

		Рижченко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – ЛрЗ	Арк.
		Годлевський Ю.О				
Змн.	Арк.	№ докум.	Підпис	Дата		5

```
y_true = np.array(y_true_list)
y_pred = np.array(y_pred_list)
print(classification_report(y_true, y_pred, target_names=class_names))

cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

model.save('cats_dogs_cnn_model.keras')
```

Результат виконання:

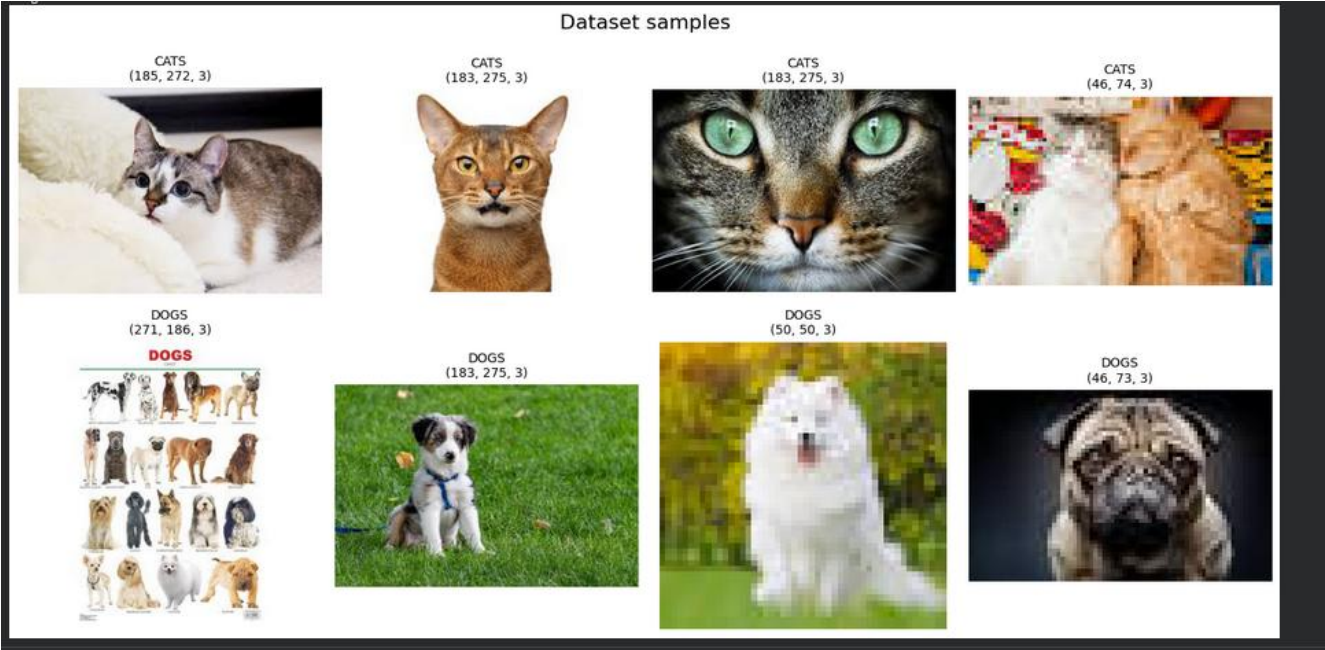


Рис. 1. Зразки зображень датасету (cats та dogs).

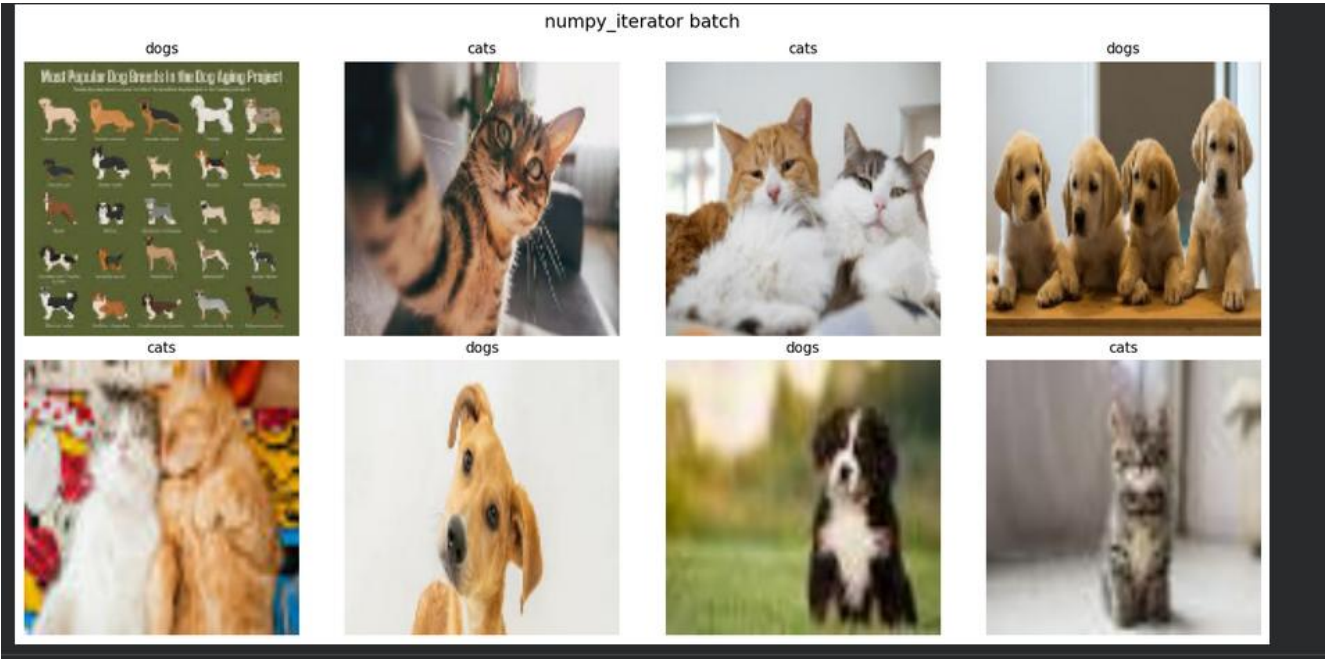


Рис. 2. Нормалізований батч з numpy_iterator.

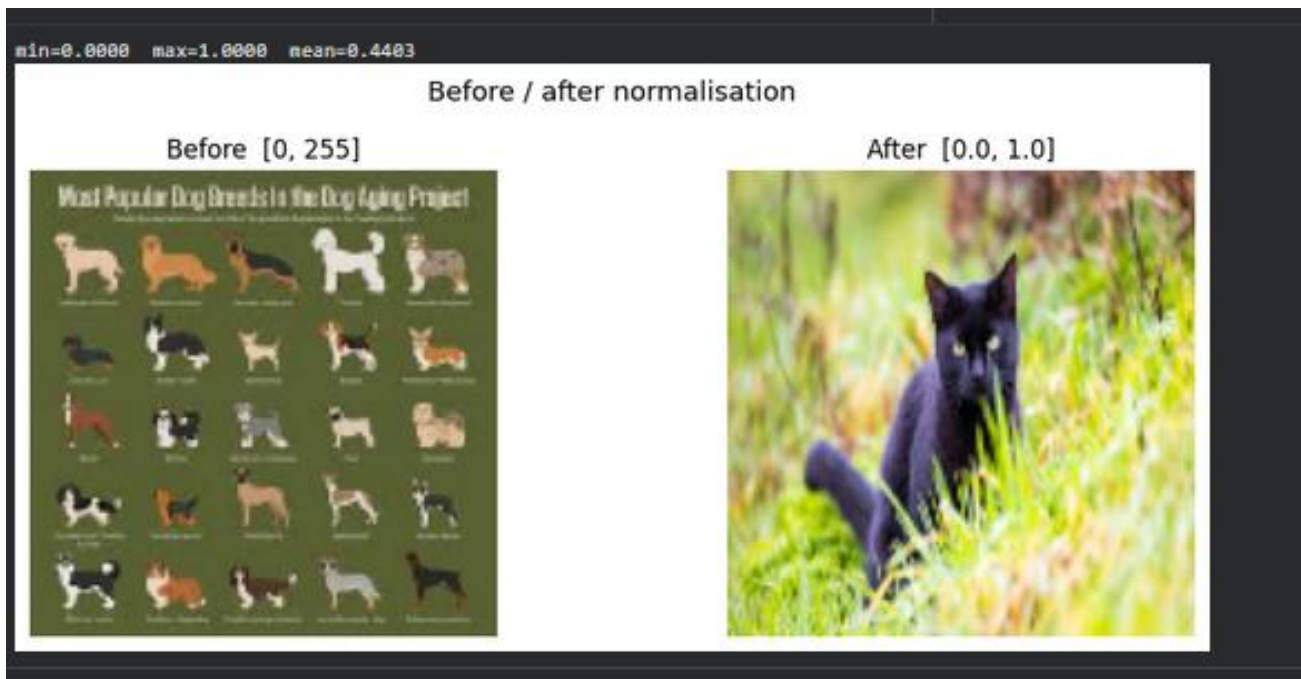


Рис. 3. Порівняння зображення до та після нормалізації.

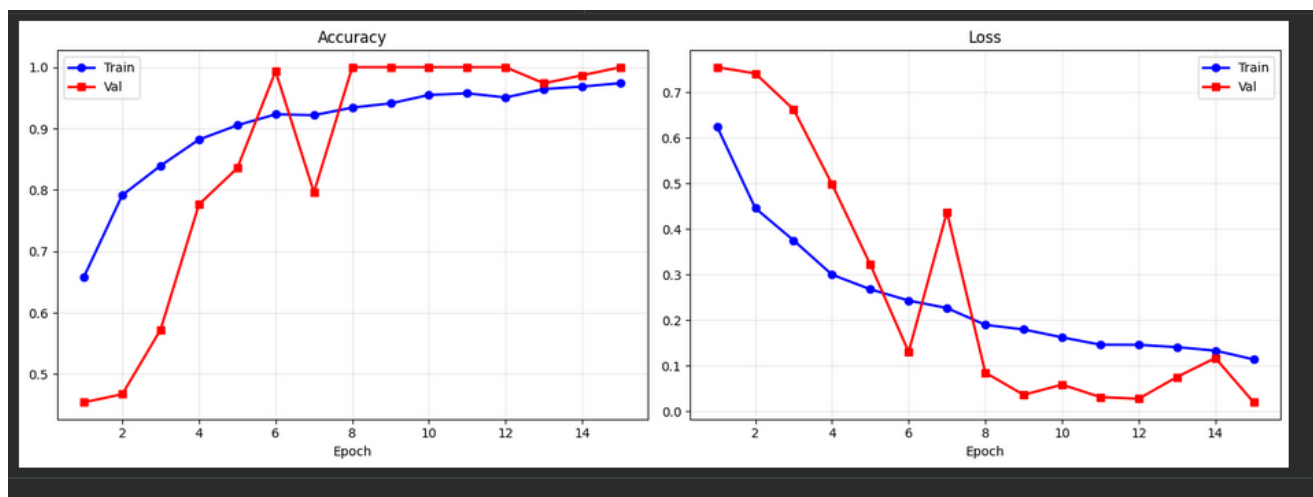


Рис. 4. Графіки Accuracy та Loss за епохами навчання.

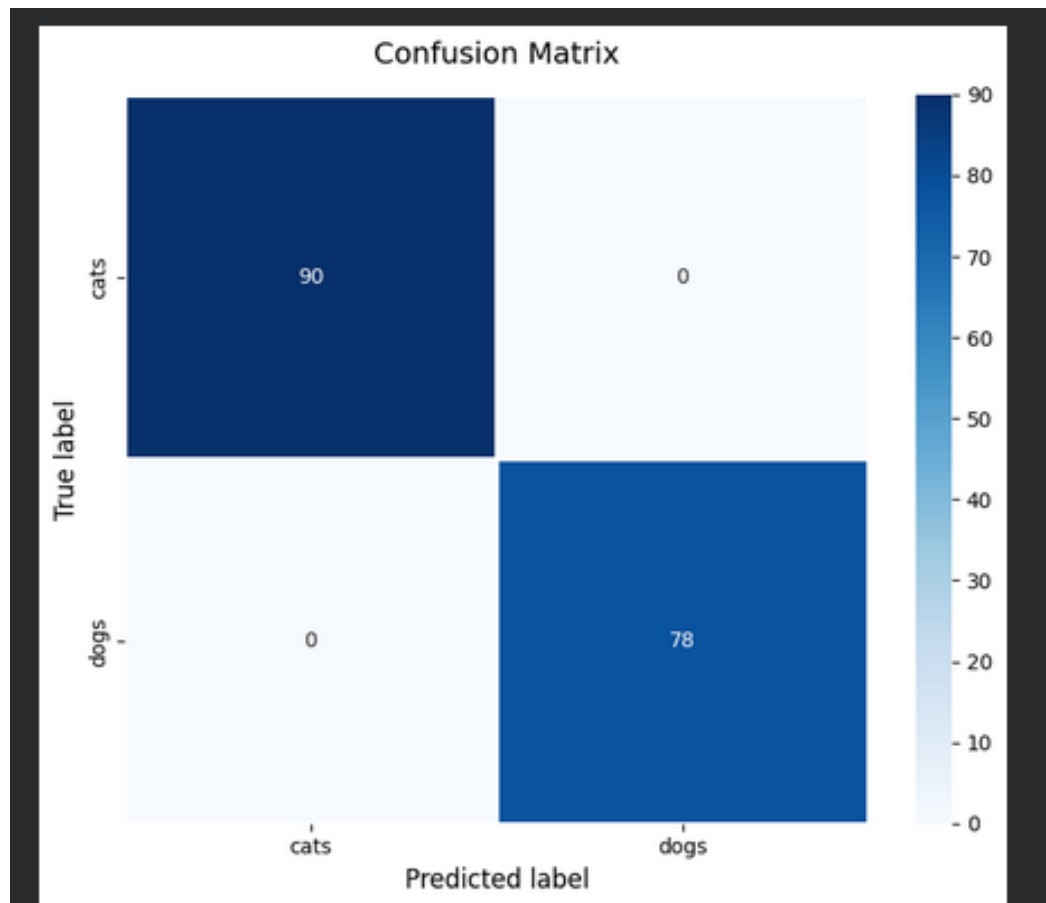


Рис. 5. Confusion Matrix.

Console output:

```
TensorFlow: 2.20.0
GPU: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
cats: 519 images
dogs: 530 images
DATASET_DIR: /content/dataset
file: image (24).jpeg
shape: (185, 272, 3)
dtype: uint8
range: 0 - 255
extensions: {'jpeg': 1049}
kept: 1049, removed: 0
Found 1049 files belonging to 2 classes.
classes: ['cats', 'dogs']
batches: 132
images: (8, 128, 128, 3)
labels: (8, 1)
unique labels: [1 0]
shape: (8, 128, 128, 3)
dtype: float32
range: [0.0, 255.0]
first 8 labels: [1 0 0 1 0 1 1 0]
min=0.0000 max=1.0000 mean=0.4403
total: 132 batches -> train: 92, val: 19, test: 21
train: 729 samples | cats=360 dogs=369
val: 152 samples | cats=69 dogs=83
test: 168 samples | cats=90 dogs=78
```

		Риженко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Лр3	Арк.
		Годлевський Ю.О				8
Змн.	Арк.	№ докум.	Підпис	Дата		

Epoch 1/15: accuracy: 0.6584 - loss: 0.6255 - val_accuracy: 0.4539 - val_loss: 0.7555
 Epoch 2/15: accuracy: 0.7915 - loss: 0.4460 - val_accuracy: 0.4671 - val_loss: 0.7418
 Epoch 3/15: accuracy: 0.8395 - loss: 0.3751 - val_accuracy: 0.5724 - val_loss: 0.6625
 Epoch 4/15: accuracy: 0.8820 - loss: 0.2999 - val_accuracy: 0.7763 - val_loss: 0.4987
 Epoch 5/15: accuracy: 0.9053 - loss: 0.2678 - val_accuracy: 0.8355 - val_loss: 0.3222
 Epoch 6/15: accuracy: 0.9232 - loss: 0.2431 - val_accuracy: 0.9934 - val_loss: 0.1304
 Epoch 7/15: accuracy: 0.9218 - loss: 0.2266 - val_accuracy: 0.7961 - val_loss: 0.4375
 Epoch 8/15: accuracy: 0.9342 - loss: 0.1897 - val_accuracy: 1.0000 - val_loss: 0.0849
 Epoch 9/15: accuracy: 0.9410 - loss: 0.1796 - val_accuracy: 1.0000 - val_loss: 0.0360
 Epoch 10/15: accuracy: 0.9547 - loss: 0.1623 - val_accuracy: 1.0000 - val_loss: 0.0585
 Epoch 11/15: accuracy: 0.9575 - loss: 0.1461 - val_accuracy: 1.0000 - val_loss: 0.0311
 Epoch 12/15: accuracy: 0.9506 - loss: 0.1459 - val_accuracy: 1.0000 - val_loss: 0.0274
 Epoch 13/15: accuracy: 0.9643 - loss: 0.1408 - val_accuracy: 0.9737 - val_loss: 0.0754
 Epoch 14/15: accuracy: 0.9684 - loss: 0.1332 - val_accuracy: 0.9868 - val_loss: 0.1169
 Epoch 15/15: accuracy: 0.9739 - loss: 0.1139 - val_accuracy: 1.0000 - val_loss: 0.0196
 Restoring model weights from the end of the best epoch: 15.
 21/21 - accuracy: 1.0000 - loss: 0.0256
 test accuracy: 100.00% loss: 0.0256

	precision	recall	f1-score	support
cats	1.00	1.00	1.00	90
dogs	1.00	1.00	1.00	78
accuracy			1.00	168
macro avg	1.00	1.00	1.00	168
weighted avg	1.00	1.00	1.00	168

168/168 correct (100.0%)

Посилання на репозиторій: <https://github.com/JanRizhenko/Neural-Networks>

Посилання на Google Collab: <https://drive.google.com/file/d/1hvcU88JyL-GeU65K9t6Lo-czU3rQ3Bggb/view?usp=sharing>

Висновок: У ході лабораторної роботи було реалізовано класифікацію зображень двох класів (коти та собаки) за допомогою згорткової нейронної мережі (CNN). Датасет складався з 519 зображень котів та 530 зображень собак. Дані були завантажені з Google Drive, зчитані та відображені, перевірені на допустимі розширення (залишено лише jpeg). Датасет нормалізовано до діапазону [0.0, 1.0] та розподілено: 70% - тренувальні, 15% - валідаційні, 15% - тестові. CNN-мережа містить чотири блоки Conv2D + BatchNormalization + MaxPooling2D, а також Dense-шари з Dropout для регуляризації. Навчання проводилось протягом 15 епох з використанням EarlyStopping та ReduceLROnPlateau. Результати тестування: точність 100%, loss 0.0256, що свідчить про ефективну роботу CNN для бінарної класифікації зображень.

		Рижченко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – ПрЗ	Арк.
		Годлевський Ю.О				
Змн.	Арк.	№ докум.	Підпис	Дата		9